CLAUDE CODE · GUÍA PRÁCTICA

Mi settings.json completo para trabajar con Claude Code

La configuración que uso todos los días: permisos, observabilidad con hooks, variables de entorno y **Agent Teams** — la función experimental para que Claude Code coordine varios agentes trabajando en equipo sobre una misma tarea.

permissions · hooks · env · CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS · statusLine

~/.claude/settings.json

```
{
  "model": "opus",
  "permissions": { /* ... */ },
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  },
  "hooks": { /* ... */ }
}
```

ANTES DE EMPEZAR

Qué vas a encontrar en esta guía

1. **El settings.json completo**
   El archivo entero, listo para copiar y adaptar a tu proyecto.
2. **Qué hace cada parte**
   model, permissions, env y hooks explicados uno por uno, con ejemplos.
3. **Agent Teams a fondo**
   Qué es CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS, cómo se activa, cómo coordina varios agentes y cuándo conviene usarlo.
4. **Extras que ayudan en el día a día**
   statusLine, outputStyle, atribución de commits y dónde vive cada nivel de configuración.

Dónde va este archivo

Guárdalo como **~/.claude/settings.json** si quieres que aplique en todos tus proyectos, o como **.claude/settings.json** dentro de un repo para que aplique solo ahí (y puedas versionarlo con el equipo). Claude Code también lee **.claude/settings.local.json** para overrides personales que no se suben a git.

Precedencia, de mayor a menor prioridad

1. Configuración gestionada por tu organización → 2. `claude --settings` → 3. **.claude/settings.local.json** → 4. **.claude/settings.json** (del proyecto) → 5. **~/.claude/settings.json** (tuyo, para todo). Si una clave se repite en varios archivos, gana la de más arriba en esta lista.

01 — EL ARCHIVO COMPLETO

settings.json completo, listo para copiar

Combina todo lo que se explica en esta guía: permisos, observabilidad con hooks (PreToolUse / PostToolUse), variables de entorno y Agent Teams. Ajusta los valores de `permissions` a tu propio flujo de trabajo antes de usarlo.

settings.json

```
{
  "model": "opus",
  "effortLevel": "high",

  "permissions": {
    "defaultMode": "ask",
    "allow": [
      "Bash(npm run *)",
      "Bash(git status)",
      "Bash(git diff *)",
      "Read(*)"
    ],
    "ask": [ "Bash(git push *)" ],
    "deny": [
      "Bash(rm -rf *)",
      "Read(./.env)"
    ],
    "additionalDirectories": [ "../shared-lib" ]
  },

  "env": {
    "DISABLE_TELEMETRY": "1",
    "BASH_DEFAULT_TIMEOUT_MS": "300000",
    "MAX_THINKING_TOKENS": "16000",
    "CLAUDE_CODE_SUBAGENT_MODEL": "sonnet",
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  },

  "teammateMode": "in-process",

  "hooks": {
    "PreToolUse": [
      { "matcher": "*", "hooks": [
        { "type": "command",
          "command": "jq -r '\"[PRE] \(.tool_name) → \(.tool_input)\"' >> ~/agent-logs/trace.log" }
      ]}
    ],
    "PostToolUse": [
      { "matcher": "*", "hooks": [
        { "type": "command",
          "command": "jq -r '\"[POST] \(.tool_name) → \(.tool_result.text)\"' >> ~/agent-logs/trace.log" }
      ]}
    ]
  },

  "statusLine": { "command": "~/.claude/status.sh" },
  "outputStyle": "Explica como un mentor senior: directo, sin relleno",
  "attribution": { "commit": "Co-authored-by: Claude <claude@anthropic.com>" },
  "cleanupPeriodDays": 30
}
```

model, effortLevel y permissions

`model` string

Define con qué modelo arranca cada sesión. Acepta el nombre completo de una versión, o un alias de familia como `"opus"` o `"sonnet"` — Claude Code resuelve el alias a la versión más reciente disponible en tu cuenta, así el settings.json no queda desactualizado cuando sale un modelo nuevo.

`effortLevel` string

Nivel de esfuerzo/razonamiento por defecto para los modelos que no tengan uno guardado. Valores típicos: **low** **medium** **high** **max** .

Más esfuerzo = respuestas más cuidadas, pero más lentas y más caras en tokens.

`permissions` object

Controla qué puede ejecutar Claude Code sin pedirte confirmación. Es la clave que más impacta tu día a día porque decide cuántas veces te va a interrumpir con un "¿puedo ejecutar esto?".

Clave	Qué hace
`defaultMode`	Modo con el que arranca la sesión: **ask** (pregunta siempre), **auto**, **plan** (solo lee y planifica) o **bypassPermissions** (no pregunta nada — úsalo con cuidado).
`allow`	Lista de acciones que se ejecutan sin preguntar. Ej: `"Bash(npm run *)"` aprueba cualquier script de npm.
`ask`	Fuerza a que SIEMPRE se pregunte, incluso si el modo activo es auto. Útil para acciones sensibles como `git push`.
`deny`	Bloquea la acción por completo, sin opción de aprobarla en el momento. Ideal para proteger `.env`, credenciales o comandos destructivos.
`additionalDirectories`	Carpetas fuera del proyecto actual a las que Claude también puede acceder (por ejemplo, una librería compartida en otro repo).

env y hooks

`env` object

Variables de entorno que Claude Code inyecta en la sesión y en todos los procesos que lanza (Bash, hooks, servidores MCP, subagentes y teammates). Es la forma de activar comportamientos experimentales o ajustar límites sin tocar tu shell.

Variable	Para qué sirve
`DISABLE_TELEMETRY`	Desactiva la telemetría que Claude Code envía por defecto.
`BASH_DEFAULT_TIMEOUT_MS`	Tiempo máximo (en ms) que espera un comando de Bash antes de cortarlo. Súbelo si trabajas con builds largos.
`MAX_THINKING_TOKENS`	Tope de tokens que el modelo puede usar para "pensar" antes de responder.
`CLAUDE_CODE_SUBAGENT_MODEL`	Modelo que usan los subagentes y teammates por defecto, en vez de heredar el del agente principal.
`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS`	Activa Agent Teams — varios Claude Code coordinados en equipo. Lo explico a fondo en la siguiente sección.

`hooks` object

Comandos que Claude Code ejecuta automáticamente en momentos puntuales de su ciclo de vida. Con `PreToolUse` y `PostToolUse` construyes un log completo de la ejecución: qué herramienta se llamó, con qué parámetros, y qué resultado obtuvo — la base de la observabilidad de agentes.

Clave	Qué hace
`matcher`	Filtra a qué herramientas aplica el hook. `"*"` = todas; también acepta nombres exactos (`"Bash"`) o expresiones regulares.
`type`	Tipo de hook. `"command"` ejecuta un script de shell; también existen tipos para llamar a un servidor MCP, un webhook HTTP o incluso otro agente.
`command`	El comando que se ejecuta. Recibe por stdin un JSON con `tool_name`, `tool_input` y, en PostToolUse, `tool_result`.

También existen hooks para otros momentos: inicio y fin de sesión, cuando cambias de directorio, cuando Claude te pide confirmación, o cuando falla una respuesta. El patrón es siempre el mismo: un evento, un matcher y un comando.

03 — A FONDO

Agent Teams: que Claude Code trabaje en equipo

Es la función más pedida de este guion. La activas con una sola variable de entorno y cambia por completo cómo delegas trabajo grande.

Cómo se activa

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Es experimental y está desactivada por defecto. Sin esta variable, Claude Code nunca arma un equipo por su cuenta.

¿Qué es exactamente?

Con Agent Teams, una sesión de Claude Code (el **lead**) puede lanzar varias instancias más de Claude Code (los **teammates**) que trabajan en paralelo, cada una con su propia ventana de contexto. No son subagentes que simplemente devuelven un resultado: los teammates se comunican directamente entre sí, se reparten tareas de una lista compartida, y pueden debatir o revisar el trabajo del otro antes de que el lead junte las conclusiones.

Cómo lo activas en la práctica

No hace falta "crear" un equipo a mano. Con la variable activada, se lo pides a Claude en lenguaje natural y él arma el equipo:

prompt

```
Lanza 3 teammates para revisar este PR:
- uno enfocado en seguridad
- uno revisando performance
- uno validando la cobertura de tests
```

03 — A FONDO

Cómo se coordina el equipo por dentro

1. **Team lead**
   La sesión principal, la que hablas tú. Reparte tareas, coordina y junta los resultados al final.

2. **Teammates**
   Instancias independientes de Claude Code, cada una con su propio contexto. Cargan el mismo CLAUDE.md, MCPs y skills del proyecto, pero no ven el historial de conversación del lead — reciben solo el prompt con el que se les lanzó.

3. **Lista de tareas compartida**
   Las tareas tienen tres estados (pendiente, en curso, completada) y pueden depender unas de otras. Un teammate termina una tarea y toma la siguiente disponible por su cuenta, o el lead se la asigna directamente.

4. **Mensajería directa**
   Cada teammate tiene su propio "buzón". Se pueden mandar mensajes entre ellos por nombre, sin pasar por ti ni por el lead — así es como debaten o se corrigen entre sí.

Modos de visualización

Modo	Cómo se ve
`in-process`	Todos los teammates corren dentro de tu misma terminal. Usas las flechas para elegir uno y Enter para ver su transcripción. Es el modo por defecto — funciona en cualquier terminal.
`tmux`	Cada teammate en su propio panel, todos visibles a la vez. Requiere tener tmux instalado.
`iterm2`	Paneles nativos de iTerm2 (requiere el CLI `it2`).

Se elige con la clave `teammateMode` en el settings.json, o al vuelo con `claude --teammate-mode`.

Agent Teams vs. subagentes

Un subagente hace un trabajo puntual y te devuelve el resultado — el que coordina todo sigue siendo el agente principal. En Agent Teams, los teammates se autoorganizan: hablan entre ellos, comparten hallazgos y se reparten tareas solos. Usa subagentes para algo puntual y rápido; usa Agent Teams cuando el trabajo se beneficia de que varias perspectivas discutan entre sí — revisiones de código, investigar un bug con varias hipótesis, o construir módulos independientes en paralelo.

Costo, límites y buenas prácticas

⚠ Consume muchos más tokens

Cada teammate es una instancia completa de Claude Code con su propia ventana de contexto. Un equipo de 3 a 5 personas escala el gasto de tokens de forma prácticamente lineal frente a una sola sesión. Resérvalo para tareas donde la exploración en paralelo realmente aporta valor — no para trabajo secuencial o rutinario.

Limitaciones a tener en cuenta

`Sin reanudar sesión`	`/resume` no restaura los teammates en modo in-process; si retomas una sesión, pídele al lead que los vuelva a lanzar.
`Sin equipos anidados`	Un teammate no puede lanzar sus propios teammates — solo el lead arma el equipo.
`Un equipo por sesión`	No se pueden crear equipos adicionales ni compartir uno entre sesiones distintas.
`Permisos fijos al lanzar`	Todos los teammates arrancan con el mismo modo de permisos que el lead. Se pueden cambiar después, pero no al momento de lanzarlos.
`Choques de archivos`	Si dos teammates editan el mismo archivo, uno sobrescribe al otro. Repárteles archivos distintos.

Buenas prácticas

- → **Arranca con 3 a 5 teammates.** Es el punto donde el trabajo en paralelo compensa la coordinación extra, sin volverse difícil de seguir.
- → **Dale contexto explícito a cada uno.** Los teammates no heredan tu conversación previa — solo lo que le escribas en el prompt de lanzamiento.
- → **Empieza con investigación, no con código.** Revisar un PR o investigar un bug es más fácil de coordinar que hacer que varios agentes escriban código a la vez.
- → **Supervisa el avance.** Dejar el equipo corriendo sin revisarlo durante mucho tiempo aumenta el riesgo de trabajo desperdiciado.

04 — EXTRAS DEL DÍA A DÍA

Otras claves que vale la pena conocer

`statusLine` `object`

Corre tu propio script y lo muestra como una barra de estado debajo del prompt. Útil para ver, por ejemplo, la rama de git activa o cuánto contexto llevas usado, sin tener que preguntarlo.

`outputStyle` `string`

Cambia el tono y el formato con el que responde Claude — por ejemplo, que sea más directo y menos explicativo, o que siga una estructura fija en cada respuesta.

`attribution` `object`

Controla el texto que Claude Code agrega a tus commits y pull requests (por ejemplo, el trailer `Co-authored-by`). Reemplaza a la clave antigua `includeCoAuthoredBy`, que quedó obsoleta.

`cleanupPeriodDays` `number`

Cuántos días guarda Claude Code las transcripciones de sesiones antes de borrarlas automáticamente.

> **Recuerda**
>
> Un settings.json no necesita tener todo lo de esta guía para ser útil. Empieza por **permissions** y los **hooks** de observabilidad, y ve sumando el resto (como Agent Teams) cuando el proyecto realmente lo pida.